Dream Park — Rapport des tests et de validation

Projet : Dream Park — Système de gestion de parking

Formation : Licence 3 MIASHS, parcours Informatique et SHS

Établissement : Université Toulouse — Jean-Jaurès

Date de vérification : 9 novembre 2026

Framework de test : unittest (bibliothèque standard Python)

1. Objet du document

Ce document explique l'organisation et le rôle des tests automatisés de Dream Park. Il présente les scénarios vérifiés, les méthodes de lancement, les résultats observés et la différence entre une validation automatisée et une vérification manuelle de l'interface.

L'application suit une architecture MVC : les tests couvrent les objets métier (Class/), leurs interactions (parking, tickets, services, persistance) et les actions du contrôleur (Controleur/). Les vues Tkinter sont principalement vérifiées via des substituts (MagicMock) et des tests complémentaires du comportement graphique.

2. Résultat de la campagne de tests

Indicateur	Résultat
Fichiers de tests	21
Tests exécutés	205
Tests réussis	205
Échecs (FAIL)	0
Erreurs (ERROR)	0

Commande exécutée depuis la racine du projet :

```
python -m unittest discover -s Tests -p "test_*.py" -v
```

Résultat synthétique observé :

```
----------------------------------------------------------------------
Ran 205 tests

OK
```

La durée d'exécution dépend de la machine. Un résultat OK confirme les scénarios couverts par les tests, pas l'absence de tout défaut possible.

3. Inventaire de tous les tests

Le nombre de tests est calculé à partir des suites unittest réellement découvertes dans les 21 modules.

Fichier	Tests	Fonctionnalités vérifiées
test_abonnements.py	26	Souscription, changement de formule, résiliation, cartes et validations du contrôleur
test_acces.py	33	Entrées, sorties, refus, services et parcours d'accès aux véhicules
test_administrateur.py	9	Modèle administrateur, ouverture/retour d'espace et accès aux statistiques

Fichier	Tests	Fonctionnalités vérifiées
test_audit_interface.py	5	Affichage de l'heure de livraison et gestion des fenêtres Tkinter
test_camera.py	1	Lecture des dimensions et de l'immatriculation par la caméra simulée
test_corrections_integration.py	5	Raccordements de l'interface et sauvegardes après opérations
test_corrections_securite.py	15	Doublons de véhicules, cohérence des objets, maintenance et retour
test_deux_acces.py	7	Indépendance des bornes, entrée/sortie croisées et sauvegarde des accès
test_identifiants_lisibles.py	7	Numéros de carte à 8 chiffres et références de ticket à 6 caractères
test_interface_statistiques.py	8	Ouverture des statistiques et export TXT/HTML, y compris les erreurs
test_interface_voiturier.py	11	Liste des livraisons, confirmation, refus, exécution et sauvegarde
test_panneau.py	2	Affichage du nombre de places avant et après un stationnement
test_place.py	5	Occupation, libération, et refus d'une place déjà utilisée
test_placement.py	1	Fin de stationnement, horodatage et libération de place
test_reference_ticket.py	11	Validité, expiration, réentrée et récupération par référence courte
test_sauvegarde.py	15	Création JSON, rechargement, propriétaires, abonnements, historique et tickets
test_statistique.py	22	Fréquentation, durée des stationnements, niveaux, places et services
test_teleporteur.py	10	Stationnement, maintenance/entretien, retour et livraison simulée
test_voiturier.py	8	Vérification et exécution des livraisons ordinaires et garanties
test_vue_statistiques_html.py	2	Génération et écriture d'un rapport HTML
test_vue_statistiques_texte.py	2	Génération et écriture d'un rapport TXT
Total	205	21 fichiers

4. Stratégie de validation

4.1 Tests unitaires des classes métier

Les tests créent des instances de Place, Parking, Placement, Voiture, Client, Abonnement, Acces, Teleporteur, Voiturier, Administrateur ou Statistique, puis utilisent assertEqual, assertTrue, assertFalse, assertIs et d'autres assertions pour confronter le résultat obtenu au résultat attendu.

Exemple : une place libre doit devenir occupée après stationnement, puis redevenir libre après la fin du placement ; une entrée doit être refusée si un véhicule est déjà stationné avec la même immatriculation.

4.2 Tests d'intégration

Ces tests enchaînent plusieurs classes du modèle pour vérifier un parcours complet : entrée → affectation d'une place → ticket → sortie, ou demande de livraison → exécution → libération de place.

Ils contrôlent également la conservation d'une même voiture dans le registre, la cohérence des propriétaires, la réutilisation des places et le renouvellement des références de tickets après une nouvelle entrée.

4.3 Tests des actions du contrôleur

Les tests des interfaces créent généralement ControleurPrincipal sans démarrer toute l'interface et remplacent les fenêtres par des objets MagicMock. Cela permet de contrôler la liaison des boutons, les appels de méthodes, la validation des formulaires, la gestion des erreurs et les appels à sauvegarder_donnees() sans bloquer les tests sur des fenêtres.

Certaines boîtes de dialogue de confirmation et fenêtres sont remplacées temporairement avec patch afin de simuler les clics « Oui » ou « Non ».

4.4 Tests de persistance isolés

Les tests de test_sauvegarde.py utilisent TemporaryDirectory pour créer un fichier JSON indépendant de donnees/dreampark.json. Ils vérifient l'enregistrement, le rechargement et la conservation des liens entre les voitures, les propriétaires, les places et les placements.

Cela garantit que l'exécution de ces tests n'écrase pas les données réelles du parking.

5. Scénarios clés détaillés

5.1 Entrée et attribution d'une place

Situation : un véhicule de dimensions compatibles se présente à une borne.

Action : appel de lancer_procedure_entree via Acces.

Résultat attendu : une place est affectée, le placement est actif, la voiture est présente dans le parking, le compteur de places libres diminue et un ticket est généré.

Cas négatifs contrôlés : dimensions invalides, parking complet, place inadaptée, choix d'abonnement ou de paiement invalide, service non reconnu et double stationnement.

5.2 Référence de ticket et sortie

Situation : un client possède un ticket d'entrée.

Action : saisie de la référence hexadécimale de six caractères.

Résultat attendu : la bonne voiture est récupérée, le placement est terminé et la place est libérée.

La suite vérifie aussi les références absentes, incorrectes et expirées, les espaces ou minuscules, ainsi que la génération d'une nouvelle référence après réentrée.

5.3 Deux accès indépendants

Deux instances distinctes Acces(1) et Acces(2) possèdent chacune leur caméra, leur borne de tickets et leurs téléporteurs. Elles partagent le même objet Parking.

Les sept tests de test_deux_acces.py vérifient :

1. l'existence d'équipements distincts ;
2. le rejet d'un numéro d'accès autre que 1 ou 2 ;
3. une entrée à la borne 1 suivie d'une sortie à la borne 2 ;
4. le parcours inverse ;
5. l'égalité des nombres de places libres affichés sur les deux accès ;
6. la conservation des accès d'entrée/sortie dans le JSON ;
7. la compatibilité des anciennes sauvegardes ne contenant pas les nouveaux champs.

Les champs accesEntree et accesSortie de Placement valent 1, 2 ou None lorsqu'ils sont inconnus ou non applicables.

5.4 Abonnements et numéros de cartes

Les tests vérifient la souscription à une formule classique, l'évolution vers une formule Super-abonné, la résiliation, la conservation du numéro attribué et la production d'un numéro de carte à huit chiffres, en évitant les collisions lors de son attribution.

Le fichier JSON de démonstration a par ailleurs fait l'objet d'une correction ponctuelle de deux fiches (une carte dupliquée et une carte manquante) ; cette correction des données existantes doit être distinguée de la logique

automatique testée.

5.5 Maintenance, entretien et livraison

Pour les services, les tests portent sur l'enregistrement d'une demande, sa modification, le départ du véhicule en maintenance ou entretien, son retour, la création d'un nouveau placement et les modalités de livraison.

Les tests du Voiturier refusent une livraison lorsque le véhicule est inconnu, sans placement actif, sans demande de livraison ou sans adresse/heure. En cas de livraison validée, la place est libérée et le placement est clos.

5.6 Statistiques et exports

Les tests vérifient le nombre de passages, les véhicules les plus fréquents, l'activité par service, les durées moyenne et maximale, les niveaux et places les plus utilisés.

Les tests d'interface vérifient aussi la génération de fichiers TXT et HTML, l'annulation par l'utilisateur et le traitement d'une erreur d'écriture.

5.7 Sauvegarde JSON et redémarrage

Après une sauvegarde et un rechargement dans un nouvel objet Parking, les tests comparent les clients, numéros de carte, abonnements, propriétaires, véhicules, places, états d'occupation, historiques, références de tickets et demandes de livraison.

Une référence de ticket active reste utilisable après rechargement ; une référence clôturée est considérée comme expirée.

6. Exécuter les tests

Prérequis : ouvrir un terminal à la racine DreamPark.

Tous les tests, affichage détaillé :

```
python -m unittest discover -s Tests -p "test_*.py" -v
```

Tous les tests, sortie courte :

```
python -m unittest discover -s Tests -p "test_*.py" -q
```

Uniquement les tests des deux accès :

```
python -m unittest Tests.test_deux_acces -v
```

Uniquement les tests de sauvegarde :

```
python -m unittest Tests.test_sauvegarde -v
```

Uniquement les tests du voiturier :

```
python -m unittest Tests.test_voiturier Tests.test_interface_voiturier -v
```

En cas d'erreur d'import, vérifier que la commande est bien exécutée à la racine du projet et que les dossiers Class, Controleur, Vue et Tests sont présents.

7. Vérifications manuelles recommandées

Les tests automatisés ne remplacent pas complètement une inspection de l'interface graphique. Avant la démonstration, effectuer les vérifications suivantes :

- Identification : afficher la fenêtre, ouvrir les espaces Voiturier et Administrateur, puis revenir à l'identification.
- Entrée : enregistrer une voiture depuis la borne 1, vérifier le ticket et l'actualisation des deux panneaux.
- Sortie croisée : récupérer cette voiture depuis la borne 2 grâce aux six caractères, vérifier le retour de la place à l'état libre.
- Abonnement : essayer une souscription puis une reconnexion avec le numéro à huit chiffres.

- Livraison : demander une livraison depuis le compte client, la retrouver dans la liste du Voiturier et la confirmer.
- Statistiques : ouvrir l'espace Administrateur et exporter un rapport TXT puis HTML.
- Redémarrage : fermer l'application normalement, la relancer et constater que les opérations enregistrées sont conservées.

Précaution : ces vérifications modifient le fichier donnees/dreampark.json ; effectuer une copie des données avant une démonstration si l'état initial doit être conservé.

8. Limites de la couverture

Les 205 tests contrôlent les principaux traitements et plusieurs parcours intégrés, mais ne prouvent pas le bon fonctionnement sur tout ordinateur ou avec plusieurs utilisateurs simultanés. Les équipements matériels, le paiement bancaire, l'authentification professionnelle sécurisée, l'exécution automatique à une heure donnée et la gestion réelle d'un parking externe ne sont pas implémentés dans la simulation.

La validation graphique doit être reproduite sur la machine de soutenance, notamment pour confirmer la disposition des fenêtres et les boutons.

9. Bilan

La campagne de tests exécutée sur la version finale disponible a retourné 205 tests réussis sur 205, sans erreur ni échec. La suite est reproductible avec unittest et couvre les classes métier, les interactions principales, les contrôles du gestionnaire, les interfaces simulées et les opérations de persistance. Le code du projet et les scénarios détaillés restent disponibles dans le dossier Tests/.